قسم التوثيق الهندسي: المشاكل التقنية واستراتيجيات الحل

المشروع: Cloud Identity & Access Management System (IAM)
إعداد الطالب: Mohammed Abdulmalik Zaid Salem Al-Hattami
التقنيات: PL/SQL | Oracle Database 21c (XE) | Oracle APEX 21c
التخصص: الأمن السيبراني (Cyber Security)

18	10	100%	PL/SQL
تحدياً تقنياً موثقاً	جداول مترابطة	معدل المعالجة والتعافي	تشفير وأمان القواعد

السجل التفصيلي للتحديات التقنية ومعالجتها الهندسيّة (المجموعة الأولى)

【1】 تعارض كلمة "User" كاسم جدول رئيسي في قاعدة البيانات

`ORA-00903: invalid table name`

وصف المشكلة: واجهتني هذه المشكلة في بداية مرحلة بناء الهيكلية (DDL)، حيث شرعت في إنشاء جدول المستخدمين باسم User لتخزين معلومات الحسابات، إلا أن المحرك رفض تنفيذ الأمر تماماً.

السبب الجذري: الكلمة USER هي كلمة محجوزة (Reserved Word) في بيئة Oracle SQL وتُستخدم كدالة نظام ترجع اسم المستخدم الحالي.

كيف حللتها: قمت بالتحكم في معالج الأسماء عبر استخدام علامات التنصيص المزدوجة "User" في جميع السكربتات والاستعلامات، مما يجبر المحرك على معاملتها كاسم كائن محدد (Case-Sensitive Identifier) بدلاً من كلمة محجوزة.

الإثبات والنتيجة: نجاح تنفيذ السكربت وإنشاء الجدول بنجاح وضمان عدم حصول تعارض أثناء الاستعلامات المفصلة.

الدرس المستفاد: أدركت أهمية مراجعة قائمة الكلمات المحجوزة في Oracle أو اعتماد بادئات موحدة مثل APP_USERS لضمان النظافة المعمارية.

【2】 التعارض المعماري لكلمة "resource" كاسم عمود في الصلاحيات

`ORA-00904: invalid identifier`

وصف المشكلة: واجهتني أثناء تصميم جدول الصلاحيات PERMISSIONS، حيث أردت تحديد المورد المستهدف للصلاحية فسميت العمود resource، فتوقف إنشاء الجدول والمنظور المقترن به.

السبب الجذري: الكلمة RESOURCE تعتبر دوراً نظامياً محجوزاً (System Role) في Oracle لخلق الكائنات، ولا يمكن استخدامها كمعرف عمود مباشر.

كيف حللتها: قمت بإعادة إعادة هندسة تسمية العقول وبنيتها، وتغيير المسمى رسمياً إلى resource_name في جدول الصلاحيات وفي كافة العروض (Views) والروابط المقترنة.

الإثبات والنتيجة: تم قبول بناء جدول الصلاحيات واستطعت إنشاء العروض التجميعية من دون أي أخطاء في التعرف على الأعمدة.

الدرس المستفاد: استخدام دلالات صريحة ومكونة من كلمتين لأسماء الأعمدة (مثل resource_name) يحمي البنية من التعارضات.

【3】 تعذر التعرف على دالة التشفير STANDARD_HASH في المادحات

`PLS-00201: identifier 'STANDARD_HASH' must be declared`

وصف المشكلة: واجهتني هذه المشكلة عند كتابة مادح الإدخال (Trigger) لتشفير كلمات المرور تلقائياً قبل الحفظ، حيث رفض مترجم PL/SQL التعرف على الدالة STANDARD_HASH.

السبب الجذري: الحساب الخاص بالتطبيق داخل بيئة Oracle APEX لا يملك صلاحيات المباشرة للوصول إلى الدالة في نطاق التنفيذ المباشر ضمن الـ Triggers.

كيف حللتها: استخدمت مؤقتاً DBMS_UTILITY.GET_HASH_VALUE، ثم انتقلت للحل الأمني الأمثل بمنح صلاحيات التنفيذ لحزمة DBMS_CRYPTO من حساب المباشر.

الإثبات والنتيجة: تم تجميع المادح (Trigger Compile) بنجاح وبدون أي أخطاء ترجمية.

الدرس المستفاد: التفريق بين الصلاحيات الممنوحة عبر الأدوار (Roles) والصلاحيات الممنوحة مباشرة (Direct Grants) في بيئة PL/SQL.

【4】 فشل المادح (Trigger) وتوقف سلسلة إدخال البيانات المعتمدة

```
ORA-04098: trigger 'TRG_HASH_PASSWORD' is invalid
```

وصف المشكلة: واجهتني مشكلة انقطاع كلي في تنفيذ سكربت التهيئة، حيث فشلت عمليات إدخال المستخدمين الجدد وتوقفت عملية ربط الأدوار USER_ROLES تماماً.
السبب الجذري: المادح TRG_HASH_PASSWORD دخل في حالة عدم صلاحية (Invalid State) بسبب خطأ الترجمة السابق، مما منع أي عملية INSERT على جدول المستخدمين.
كيف حللتها: أصلحت كود المادح، ونفذت أمر إعادة التجميع الصريح ALTER TRIGGER TRG_HASH_PASSWORD COMPILE; ثم أعدت تنفيذ السكربت بالكامل.

الإثبات والنتيجة: عودة المادح إلى حالة VALID وتنفيذ كافة عمليات الإدخال التتابعية دون أخطاء.

الدرس المستفاد: معالجة الأخطاء من منشئها الأول وتفقد حالة الكائنات (Status of DB Objects) قبل البدء بضخ البيانات.

【5】 خطأ إنشاء مستخدم التطبيق في الحاوية الجذرية CDB بدلاً من PDB

```
ORA-65096: invalid common user or role name
```

وصف المشكلة: واجهتني عند إعداد البيئة وتأسيس مستخدم قاعدة البيانات للمشروع MY_APP_DB عبر سطر الأوامر، حيث رفض النظام إنشاء الحساب.
السبب الجذري: الاتصال المباشر كان موجهاً إلى الحاوية الجذرية (CDB$ROOT) التي تتطلب أن تبدأ أسماء المستخدمين بـ c## ولا تقبل المستخدمين المحليين.
كيف حللتها: قمت بالتحويل إلى الحاوية القابلة للوصل بواسطة الأمر: ALTER SESSION SET CONTAINER = XEPDB1; ثم أعدت أمر إنشاء الحساب بشكل طبيعي.

الإثبات والنتيجة: إنشاء المستخدم وتخصيص مساحة التخزين (Tablespace) له داخل XEPDB1 بنجاح.

الدرس المستفاد: فهم المعمارية متعددة المستأجرين (Multitenant Architecture) في Oracle 21c ضروري للتطوير السليم.

【6】 فشل الاتصال بقاعدة البيانات عبر SQL*Plus وتوقف المحول

```
ORA-12560: TNS:protocol adapter error
```

وصف المشكلة: واجهتني عند محاولة الدخول إلى محث الأوامر sqlplus / as sysdba للقيام بالعمليات الإدارية، حيث ظهرت رسالة فشل محول بروتوكول الاتصال.
السبب الجذري: بيئة موجه الأوامر (CMD) لم تكن تحتوي على متغير البيئة ORACLE_SID المحدد لاسم المقتطع الفعال.
كيف حللتها: قمت بضبط المتغير محلياً في نافذة الأوامر بنموذج: set ORACLE_SID=XE وتأكدت من عمل خدمة OracleServiceXE في خدمات النظام.

الإثبات والنتيجة: تم الاتصال الفوري بقاعدة البيانات بصلاحيات SYSDBA بنجاح.

الدرس المستفاد: التحقق دائماً من متغيرات البيئة والحالة التشغيلية لخدمات قاعدة البيانات عند العمل مع الأدوات المباشرة.

【7】 تقييد الصلاحيات الأمنية على حزمة التشفير Advanced Cryptography

```
PLS-00201: identifier 'DBMS_CRYPTO' must be declared
```

وصف المشكلة: واجهتني أثناء ترقية آلية تشفير كلمات المرور باستخدام التشفير القوي SHA-256، حيث رفض المترجم استخدام الحزمة.
السبب الجذري: الحزمة المعقدة DBMS_CRYPTO مملوكة لـ SYS ولا تُمنح صلاحيات تنفيذها للمستخدمين العاديين افتراضياً لأسباب أمنية.
كيف حللتها: دخلت كـ SYSDBA داخل الحاوية XEPDB1 ونفذت منحاً صريحاً: GRANT EXECUTE ON DBMS_CRYPTO TO MY_APP_DB;.

الإثبات والنتيجة: تمكن مخطط التطبيق من الاستدعاء المباشر لخوارزميات التشفير المعقدة بداخل الـ PL/SQL.

الدرس المستفاد: مبدأ الصلاحيات الأدنى (Least Privilege) يستوجب منح صلاحيات الحزم الحساسة يدوياً وبدقة.

【8】 انحراف جلسة APEX والارتباط بمخطط مستخدم غير صحيح

`المشكلة: استعلام SELECT USER يُرجع APEX_PUBLIC_USER`

وصف المشكلة: واجهتني أثناء تنفيذ الاستعلامات داخل واجهة APEX، حيث تبين أن العمليات تشير إلى المستخدم الافتراضي للواجهة بدلاً من مخطط المشروع.
السبب الجذري: عدم ربط مساحة العمل (Workspace) بالمخطط البرمجي الصحيح MY_APP_DB بشكل كامل في إعدادات البيئة.
كيف حللتها: دخلت إلى إعدادات إدارة مساحة العمل (Internal Administration)، وأعدت تعيين المخطط الأساسي (Schema Provisioning) ليكون MY_APP_DB.

الإثبات والنتيجة: إرجاع القيمة MY_APP_DB عند استعلام SELECT USER FROM DUAL; من داخل بيئة APEX.

الدرس المستفاد: التأكد من فصل سياق التنفيذ (Execution Context) بين محرك التطبيق ومخطط البيانات الشارح.

【9】 انتهاك القيود التكاملية المرجعية أثناء ترتيب عمليات الإدخال

`ORA-02291: integrity constraint violated - parent key not found`

وصف المشكلة: واجهتني عند تشغيل السكربت الشامل لبناء وتعبئة البيانات، حيث توقف الإدخال في جدول العلاقات USER_ROLES.
السبب الجذري: محاولة تعيين أدواره لمستخدمين قبل إتمام إدراجهم في جدول "User" الأب، أو استخدام معرفات غير قائمة.
كيف حللتها: إعادة ترتيب تسلسل السكربت (User → Roles → User_Roles)، والاستعاضة عن القيم الثابتة بالاستعلامات الفرعية (Subqueries).

الإثبات والنتيجة: نجاح السكربت بالكامل وربط السجلات بدون أدنى تعارض في العلاقات المرجعية.

الدرس المستفاد: احترام شجرة التبعيات (Dependency Tree) في قواعد البيانات المترابطة خطوة أساسية في استقرار النظام.

【10】 فشل تنفيذ أوامر الإدخال المتعدد المستوردة من صيغ غريبة

`ORA-00933: SQL command not properly ended`

وصف المشكلة: واجهتني عند محاولة إدخال سجلات متعددة دفعة واحدة في جدول ROLE_PERMISSIONS لتسريع العملية، حيث توقف المحرك عن العمل.
السبب الجذري: استخدام بناء جملة غير مدعوم في Oracle مثل VALUES (...) ,(...) ,(...) الشائع في محركات مثل MySQL.
كيف حللتها: تحويل صيغة الإدخال إلى الصيغة القياسية لنظام Oracle باستخدام INSERT ALL أو الاعتماد على INSERT INTO ... SELECT ... FROM DUAL.

الإثبات والنتيجة: إدراج كافة الصلاحيات والأدوار بنجاح ودون أخطاء تركيبية.

الدرس المستفاد: الالتزام الصارم بالمعايير التركيبية (Syntax Standard) الخاصة بالمحرك المستهدف (Oracle Dialect).

【11】 عدم استقرار قيم الهوية المحدثة تلقائياً IDENTITY Column

`المشكلة: انزلاق قيم الـ IDs وفشل الروابط الثابتة`

وصف المشكلة: واجهتني عند كتابة سكربتات ربط المفاتيح الخارجية، حيث كانت القيم المولّدة لـ user_id تتغير وتتجاوز التسلسل الترتيبي.
السبب الجذري: أعمدة IDENTITY تعتمد على متتاليات نظام (Sequences) قد تتأثر بـ DELETE أو التراجع عن الجلسات، فلا تبدأ دائماً من 1.
كيف حللتها: ألغيت الاعتماد على الأرقام الصريحة (Hardcoded IDs) واستبدلتها بديناميكية الاستعلام الفرعي مثل: (SELECT user_id FROM "User" WHERE username='admin').

الإثبات والنتيجة: ربط العلاقات بدقة 100% بغض النظر عن القيمة الرقمية المولدة للمفتاح الرئيسي.

الدرس المستفاد: التخلي التام عن افتراض أرقام المفاتيح التلقائية وتصميم السكربتات بأسلوب مرن (Dynamic Resolution).

【12】 ضعف الأمان التشفيري وتحدي توليد قيمة Salt فريدة ديناميكياً

المشكلة: تشابه التشفير لكلمات المرور المتطابقة

وصف المشكلة: واجهتني الحاجة الأمنية لمنع هجمات جداول التشفير المسبق (Rainbow Tables)، حيث كان يتوجب دمج قيمة Salt متغيرة لكل مستخدم.
السبب الجذري: عدم وجود عمود مستقل لقيمة Salt يولد عشوائياً بدقة أجزاء من الثانية أثناء التأسيس.
كيف حللتها: عدلت المادح ليولد Salt مستنداً إلى الطابع الزمني المتقدم: TO_CHAR(SYSTIMESTAMP, 'YYYYMMDDHH24MISSFF') ودمجه مع كلمة المرور قبل الهاش.

الإثبات والنتيجة: اختلاق قيم تشفير مختلفة كلياً حتى لو أدخل مستخدمان نفس كلمة المرور.

الدرس المستفاد: تطبيق أفضل الممارسات الأمنية الحديثة في حماية البيانات الحساسة داخل قواعد البيانات.

【13】 تجاوز السعة التخزينية لعمود رمز التشفير المدمج

ORA-12899: value too large for column "password_hash"

وصف المشكلة: واجهتني فور تطبيق خوارزمية الـ Salt والهاش المزدوج، حيث رفض المحرك حفظ السجلات وظهر خطأ تجاوز السعة.
السبب الجذري: الحجم المخصص سابقاً للعمود كان VARCHAR2(64)، وهو غير كافٍ لاستيعاب ناتج خوارزمية التشفير المتقدمة مضافاً إليها التمليح.
كيف حللتها: وسعت سعة العمود عبر أمر التعديل: ALTER TABLE "User" MODIFY password_hash VARCHAR2(128);.

الإثبات والنتيجة: قبول حفظ النصوص المشفرة الطويلة المكونة من 128 حرفاً دون أي قص أو خطأ.

الدرس المستفاد: حساب الأبعاد التخزينية المخرجة من الخوارزميات الأمنية قبل تحديد أنواع البيانات والأحجام.

【14】 انقطاع الربط المرجعي لعمود التدقيق assigned_by

ORA-02291: integrity constraint (assigned_by) violated

وصف المشكلة: واجهتني أثناء إنشاء جداول التدقيق الخاص بـ USER_ROLES لمعرفة من قام بأسناد الدور، حيث توقف إسناد الدور القيادي.
السبب الجذري: محاولة جعل قيمة assigned_by تشير إلى مستخدم لم يتم إدراجه بعد في جدول الحسابات الأصلي.
كيف حللتها: قمت بأتمتة إدخال المستخدم المسؤول الأول admin بنحو مستقل، ثم استخراج user_id الخاص به لتغذية حقلي التعيين.

الإثبات والنتيجة: توثيق حقول التدقيق (Audit Fields) بنجاح وبشكل مرجعي سليم 100%.

الدرس المستفاد: معالجة العلاقات الدائرية والشخصية (Self-referencing & Audit Chains) بدقة متناهية.

【15】 تلوث قائمة جداول النظام بجداول نموذجية غير متعلقة بالمشروع

المشكلة: ظهور جداول مثل EMPLOYEES و DEPARTMENTS في التقرير

وصف المشكلة: واجهتني عند استعلام قاموس البيانات USER_TABLES لاستعراض جداول النظام للتوثيق، حيث ظهرت جداول خارجية غير تابعة للمشروع.
السبب الجذري: وجود عينات البيانات (Sample Schemas) المجهزة مسبقاً داخل بيئة Oracle XE بنفس المخطط.
كيف حللتها: صممت استعلام فلترة حصري يحدد جداول المشروع الـ 10 بوضوح: WHERE table_name IN ('User', 'ROLES', 'PERMISSIONS', ...).

الإثبات والنتيجة: استعراض جداول المشروع الـ 10 الحصرية فقط بنظافة كاملة في شاشات التقارير.

الدرس المستفاد: استخدام أدوات التصفية الفعالة لقاموس البيانات للحفاظ على عزل نطاق المشروع (Scope Isolation).

【16】 اصطدام القيود الفريدة عند إعادة تشغيل سكربت البناء

`ORA-00001: unique constraint violated`

وصف المشكلة: واجهتني أثناء المراجعة والتطوير المستمر، حيث كان ينقطع تنفيذ السكربت عند إعادته نتيجة وجود بيانات سابقة.
السبب الجذري: محاولة أدراج أسماء أدوار أو مستخدمين مسجلة سابقاً وتملك قيود عدم التكرار (Unique Constraints).
كيف حللتها: أضفت آلية التنسيق المسبق (Clean-up Section) بوضع أوامر DELETE FROM مرتبة عكسياً بحسب التبعية في مقدمة السكربت.

الإثبات والنتيجة: القدرة على إعادة تشغيل السكربت كاملاً في أي وقت بنجاح (Idempotent Execution).

الدرس المستفاد: كتابة سكربتات البيانات بقدرة التنفيذ التكراري دون التأثر بالحالة الراهنة لقاعدة البيانات.

【17】 اقتطاع الأوامر البرمجية الطويلة في واجهة SQL Commands

المشكلة: اقتطاع السكربتات وتوقف التنفيذ الجزئي

وصف المشكلة: واجهتني عند لصق سكربتات PL/SQL الضخمة والمتشابكة في نافذة SQL Commands المباشرة داخل APEX.
السبب الجذري: محرر SQL Commands المباشر يملك حداً أقصى لحجم النص المدخل في الجلسة الواحدة.
كيف حللتها: انتقلت لاستخدام وحدة SQL Scripts المخصصة لرفع وتثبيت وتحديث المجموعات البرمجية الكبيرة كملفات كاملة.

الإثبات والنتيجة: تنفيذ السكربت الشامل الخالي من الاقتطاع بنسبة 100% وإظهار نتائج التنفيذ بالتفصيل.

الدرس المستفاد: اختيار الأداة المناسبة داخل بيئة التطوير بحسب حجم ونوعية الأكواد البرمجية.

【18】 التثبت العملي والأكاديمي من كفاءة خوارزمية التشفير

المشكلة: الحاجة لإثبات أمان كلمات المرور إثباتاً قاطعاً

وصف المشكلة: واجهتني الحاجة لإثبات أن كلمات المرور لا تخزن بأي شكل كنص مجرد (Plaintext) في قاعدة البيانات لأغراض التقييم الأمني.
السبب الجذري: متطلب أكاديمي وأمني يقتضي برهان سلامة التشفير وعدم إمكانية قراءة البيانات حتى من قبل مدير النظام.
كيف حللتها: نفذت استعلاماً صريحاً: "SELECT username, password_hash FROM "User"; وتوثيق المخرجات بالصور الرقمية المشفرة.

الإثبات والنتيجة: إظهار قيم الهاش الملمحة والمعقدة ونشر الصورة في تقرير التخرج النهائي.

الدرس المستفاد: التوثيق الأمني المبرهن هو العصب الأساسي لمشاريع الأمن السيبراني.

جدول مصفوفة ملخص المشاكل والأثر الهيكلي:

رمز المشكلة	النطاق التقني	سبب الخطأ الرئيسي	نتيجة المعالجة
ORA-00903 / ORA-00904	تسمية الكائنات (DDL)	استخدام الكلمات المحجوزة	علامات التنصيص والمسميات الصريحة
PLS-00201 / ORA-04098	البرمجة والتشفير (PL/SQL)	الصلاحيات وحالة المادح المكسورة	منح EXECUTE وإعادة التجميع
ORA-65096 / ORA-12560	إدارة البيئة (DBA)	سياق الحاوية ومتغيرات البيئة	التحويل لـ XEPDB1 وضبط SID
ORA-02291 / ORA-00001	التكاملية المرجعية (DML)	ترتيب الإدخال وتكرار القيود	إعادة الترتيب وأوامر التنظيف المسبق

أولاً: الخاتمة والتوليف الهندسي

إن رحلة مواجهة وحل هذه المشاكل التقنية الثماني عشرة لم تكن مجرد سلسلة من أخطاء البرمجة والتصحيح، بل كانت المحرك الأساسي لتفهم الدقائق المعمارية لقواعد بيانات Oracle 21c وواجهة Oracle APEX. لقد أتاحت لي هذه التحديات الانتقال من الفهم النظرية المفاهيمي للأمن السيبراني إلى التطبيق المباشر القائم على حماية البيانات، إدارة الصلاحيات المتقدمة، وبناء معمارية صلبة تتحمل الأخطاء وتستجيب للمتطلبات العالية. إن نظام Cloud IAM الناتج اليوم ليس مجرد تطبيق يعمل بنجاح، بل هو بيئة آمنة، خالية من التعارضات المرجعية، ومبنية على أفضل المعايير الهندسية الأكاديمية.

ثانياً: الدرس الأكبر والتغير الفكري

لقد غيرت هذه المشاكل المتتابعة فلسفتي وتفكيري الهندسي كمطور وقواعد بيانات أمنية. تعلمت أن **"رسالة الخطأ ليست عائقاً، بل هي بوصلة توجهك نحو فهم أعمق للمحرك"**. لقد انتقلت من أسلوب المحاولة والخطأ التخميني إلى أسلوب التشخيص المنهجي المستند إلى السبب الجذري (Root Cause Analysis)، وقراءة قاموس النظام، وتتبع سجلات الأخطاء بدقة. كما أدركت أن التفكير الأمني يجب أن يرافق تصميم قاعدة البيانات منذ اللحظة الأولى لتأسيس الـ DDL وليس كمرحلة تجميعية لاحقة.

ثالثاً: كلمة شكر وتقدير أكاديمية

أتوجه بخالص الشكر والتقدير والامتنان إلى **الدكتورة/ أنهار الغيثي** على توجيهاتها الأكاديمية القيمة، ودعمها العلمي المستمر أثناء تنفيذ هذا المشروع. لقد كانت ملاحظاتها الدقيقة ورؤيتها الأكاديمية الثاقبة في مجال قواعد البيانات والأمن السيبراني الدافع الأكبر لي لتجاوز هذه العقبات التقنية، والوصول بالتقرير وهذا المشروع إلى هذا المستوى الرفيع من الاحترافية والدقة الهندسية.

توثيق التقرير النهائي – قسم الأمن السيبراني
اسم الطالب: Mohammed Abdulmalik Zaid Salem Al-Hattami

حالة التقرير: مُعتمد ومكتمل
النظام: Oracle APEX 21c / DB 21c XE